

A Front End and Intermediate Code Generator for the Classroom Language (CL)

CSC323 Compiler Construction Project Report

Submitted By

SAMEER HASSAN (01-134232-168)

HAROON MAZHAR (01-134232-121)

BSCS 6-C

Submitted To

Sir Abdul Raheem Aleem

Department of Computer Science
Bahria University Islamabad Campus
Spring 2026

Abstract

This report describes the design and implementation of a working compiler front end together with a three address code generator for the Classroom Language (CL), the toy language defined for the CSC323 Compiler Construction project in Spring 2026. The work covers all the phases that come before target code generation: a hand written grammar fed to JavaCC, an AST produced through the JJTree extension, a symbol table data structure, a separate semantic analyser class and an intermediate code generator that emits three address code in quadruple form.

CL supports four data types (`int`, `float`, `string` and `char`), assignment statements, an output statement (`outString`), a conditional loop (`loopif`) and a multi way branch (`switchFor`). The grammar was first written in the standard mathematical form, then transformed (left factoring and removal of left recursion) into an LL(1) form that JavaCC could consume directly. The semantic analyser walks the AST and rejects undeclared identifiers, duplicate declarations, type mismatches in declarations and assignments, illegal arithmetic on string or character values, type mismatches in conditional expressions and case literals that do not match the type of the switch variable. Each error message includes a line number so the user can find the problem quickly. The intermediate code generator emits quadruples of the form (`op`, `arg1`, `arg2`, `result`) into a Java array; the same array is then printed in two styles, once as the quadruple table required by the assignment and once in the more readable statement form.

Five valid programs and five invalid programs were used to verify that the compiler accepts well formed CL and produces a clear, line numbered error for each kind of problem the specification mentioned. The intermediate code produced for the loop and switch examples follows the patterns from chapter eight of the Dragon Book. All four data types, both control flow constructs and the strict same-type rule for binary operators were implemented and tested.

Contents

1	Introduction	5
1.1	Background	5
1.2	Problem Statement	5
1.3	Objectives	6
1.4	Scope	6
1.5	Report Organization	6
2	CL Language Specification	7
2.1	Overview	7
2.2	Program Structure	7
2.3	Supported Data Types	8
2.4	Supported Statements	8
2.5	Operators	9
3	Lexical and Syntax Analysis	10
3.1	Lexical Analyzer	10
3.2	Parser	11
3.3	Grammar Rules	11
3.4	Abstract Syntax Tree	13
4	Symbol Table and Semantic Analysis	14
4.1	Symbol Table	14
4.2	Semantic Checks	14
4.3	Error Examples	15
5	Intermediate Code Generation	17
5.1	Overview	17
5.2	Input Expression	17
5.3	Generated Three Address Code	17
5.4	Quadruple Representation	18
5.5	Code for Control Flow	18

6	Testing and Results	20
6.1	Testing Strategy	20
6.2	Valid Test Cases	20
6.3	Invalid Test Cases	21
6.4	Screenshots	21
7	Conclusion and Future Work	22
7.1	Conclusion	22
7.2	Learning Outcomes	22
7.3	Future Work	22
A	Source Code	25
B	Sample Input Programs	27
C	Extra Screenshots	29

List of Figures

- 3.1 AST of the simple addition CL program (text form printed by `node.dump()`). 13

List of Tables

3.1	CL tokens recognised by the lexer.	11
4.1	Symbol table after parsing the addition program.	14
5.1	Quadruple representation of $r = a + b * c - d / e$;	18
5.2	Quadruples generated for the factorial program.	19
6.1	Valid test cases.	20
6.2	Invalid test cases.	21

Chapter 1

Introduction

1.1 Background

A compiler is a piece of software that translates a program written in a high level source language into an equivalent program in a lower level target language. Internally a compiler is split into a number of phases: lexical analysis, syntax analysis, semantic analysis, intermediate code generation, optimisation and target code generation. The first four of these phases together are usually called the front end and the last two the back end. This project deals with the front end and the first part of the back end, that is, with everything up to and including the production of an intermediate representation.

Compiler construction is one of the more rewarding topics in computer science because the same ideas show up in many other tools: interpreters, linters, formatters, query engines, regular expression libraries and even IDE auto completion. Learning how a compiler is built also forces a programmer to think very carefully about how grammars are written, why some constructs are allowed in a language and why others are not.

1.2 Problem Statement

The task for this assignment was to extend the front end built in the first project milestone with a proper semantic analyser and an intermediate code generator. The semantic analyser had to use a symbol table to detect type errors (the example given in the assignment was `a = b + c` where `b` is a string and `c` is an integer) and the intermediate code generator had to emit three address code in quadruple form into a Java array, then show that array on the screen.

1.3 Objectives

- Write a JJT grammar for CL that supports the four data types `int`, `float`, `string` and `char` together with the `loopif` and `switchFor` control flow constructs.
- Build the symbol table as a separate Java class that stores the lexeme, type and value of every identifier.
- Implement the semantic analyser as a separate Java class that walks the AST and detects undeclared identifiers, duplicate declarations and type mismatches.
- Produce three address code as quadruples for every valid CL program and store the quadruples in a Java array.
- Test the compiler on a small but reasonably varied set of valid and invalid programs.

1.4 Scope

The scope is limited to the front end and to intermediate code generation. Optimisation passes, register allocation and final machine code generation are not included. The CL language itself is also small: it has no functions, no arrays, no nested control flow and no implicit type conversion. These restrictions come directly from the project specification and they make the language easier to teach without making the implementation trivial.

1.5 Report Organization

Chapter 2 describes the CL language as we have implemented it. Chapter 3 covers the lexical analyser and the parser, including the grammar transformations needed for LL(1) parsing. Chapter 4 explains the symbol table and every semantic check performed by the analyser. Chapter 5 deals with intermediate code generation. Chapter 6 lists the test programs and the output the compiler produced for each. Chapter 7 wraps up the report and suggests directions for future work.

Chapter 2

CL Language Specification

2.1 Overview

The Classroom Language (CL) is a small imperative language used only within this course. Every CL program begins with the keyword `startProgram` and ends with `endProgram`. Between these two keywords there are exactly two blocks: a `variables` block where every identifier used in the program is declared together with its type and initial value, and a `code` block that holds the executable statements. Each statement is terminated with a semicolon. Comments start with `//` and continue to the end of the line.

2.2 Program Structure

A small CL program that adds two numbers and prints the result is shown below.

```
startProgram

variables:
    int abc    = 3;
    int xyz    = 4;
    int result = 0;

code:
    result = abc + xyz;
    outString(result);

endProgram
```

A slightly more involved program that computes the factorial of 5 using `loopif` is shown next.

```
startProgram
  variables:
    int n      = 5;
    int fact = 1;
  code:
    loopif n > 0 holds
      fact = fact * n;
      n    = n - 1;
    endloop
    outString(fact);
endProgram
```

2.3 Supported Data Types

The project specification asks for four data types. Our implementation supports all of them.

- **int** : a signed integer such as 3, 0 or 42.
- **float** : a decimal number written with a dot, for example 3.14.
- **string** : a sequence of characters between double quotes, for example "hello".
- **char** : a single character between single quotes, for example 'A'.

There is no implicit type conversion. An assignment such as `int x = "hi"` is rejected, and so is `a + b` when `a` is a string and `b` is an integer.

2.4 Supported Statements

CL supports four kinds of statements:

- **Assignment.** A variable on the left receives the value of an expression on the right, for example `result = abc + xyz;`.
- **loopif.** A conditional loop. The body keeps executing as long as the relational expression after `loopif` holds. The block ends with `endloop`.

-
- **switchFor.** A multi way branch. The integer (or any other supported type) value of a variable is matched against a list of **case** literals; if none match, the **other** branch runs. The block ends with **endswitchFor**.
 - **outString.** Prints the value of an expression. Strings, characters, integers and floats are all acceptable arguments.

The project specification says **loopif** and **switchFor** are not allowed to be nested inside one another. The grammar enforces this directly: the body of a **loopif** only accepts **AssignStmt** or **OutStmt**, never another **loopif** or **switchFor**.

2.5 Operators

The arithmetic operators are **+**, **-**, ***** and **/**. They have the usual Java precedence (***** and **/** bind tighter than **+** and **-**) and are all left associative. The relational operators allowed in the condition of **loopif** are **<**, **>**, **<=**, **>=**, **==** and **<>**. A single **=** is reserved for the assignment statement only.

Chapter 3

Lexical and Syntax Analysis

3.1 Lexical Analyzer

The lexical analyser is generated by JavaCC from the token rules in our `CL.jjt` file. The lexer's job is to read the source character by character and group those characters into meaningful tokens. White space (spaces, tabs and newlines) and single line comments are skipped using JavaCC's `SKIP` directive so they never reach the parser.

Identifiers are required to start with a letter or an underscore and can be followed by letters, digits or underscores. Integer constants are sequences of one or more digits. Floating point constants are written with a dot between two non empty digit sequences. String constants are any characters between double quotes and char constants are exactly one character (with an optional backslash escape) between single quotes. All CL keywords (`startProgram`, `endProgram`, `variables`, `code`, `int`, `float`, `string`, `char`, `loopif`, `holds`, `endloop`, `switchFor`, `case`, `other`, `endswitchFor`, `outString`) are listed as their own token kinds before the `IDENT` rule so that they cannot accidentally be matched as identifiers. A small selection of the lexer rules is shown in Table 3.1.

Lexeme	Token Type
startProgram	T_START
endProgram	T_END
variables	T_VARS
code	T_CODE
int / float / string / char	T_INT / T_FLOAT_T / T_STR_T / T_CHAR_T
loopif / holds / endloop	T_LOOPIF / T_HOLDS / T_ENDLOOP
switchFor / case / other / endswitchFor	T_SWITCH / T_CASE / T_OTHER / T_ENDSWITCH
outString	T_OUTSTRING
abc, xyz, result	IDENT
3, 4, 0	INTEGER
3.14	FLOATING
"hello"	STRLIT
'A'	CHARLIT
+ - * /	PLUS / MINUS / MUL / DIV
< > <= >= == <>	LT / GT / LE / GE / EQ / NE
;	SEMI
()	LP / RP
:	COLON

Table 3.1: CL tokens recognised by the lexer.

3.2 Parser

The parser is also generated by JavaCC from the same `.jjt` file. It is a top down predictive parser: it begins at the start symbol **Program** and expands non terminals based on the next input token. JavaCC produces an LL(1) parser by default and supports the **LOOKAHEAD** directive whenever a single token is not enough to decide. Our grammar was written so that one token of lookahead is always enough, so no **LOOKAHEAD** directives are needed.

JJTree is the JavaCC extension that turns the parser into a tree builder. Each grammar production marked with **#NodeName** creates a node of that type when the production is reduced. The resulting Abstract Syntax Tree is the input to the semantic analyser and to the intermediate code generator.

3.3 Grammar Rules

The CL grammar as it appears in `CL.jjt` (after removing left recursion and factoring out common prefixes for LL(1) parsing) is summarised below.

```

Program      -> "startProgram" VarBlock CodeBlock "endProgram"

VarBlock     -> "variables" ":" ( VarDecl )*
VarDecl      -> Type IDENT "=" Literal ";"
Type         -> "int" | "float" | "string" | "char"
Literal      -> INTEGER | FLOATING | STRLIT | CHARLIT

CodeBlock    -> "code" ":" ( Stmt )*

Stmt         -> AssignStmt
              | LoopStmt
              | SwitchStmt
              | OutStmt

AssignStmt   -> IDENT "=" Expr ";"
OutStmt      -> "outString" "(" Expr ")" ";"

LoopStmt     -> "loopif" CondExpr "holds"
              ( AssignStmt | OutStmt )+
              "endloop"

SwitchStmt   -> "switchFor" "(" IDENT ")"
              ( "case" Literal ":" AssignStmt )+
              "other" ":" AssignStmt
              "endswitchFor"

CondExpr     -> Operand RelOp Operand
RelOp        -> "<" | ">" | "<=" | ">=" | "==" | "<>"
Operand      -> IDENT | INTEGER | FLOATING

Expr         -> Term ( ("+" | "-") Term )*
Term         -> Factor ( ("*" | "/" ) Factor )*
Factor       -> IDENT | INTEGER | FLOATING | CHARLIT
              | STRLIT | "(" Expr ")"

```

The two layer Expr / Term structure gives arithmetic the Java like precedence we wanted: * and / are evaluated before + and -.

3.4 Abstract Syntax Tree

Figure 3.1 shows the AST produced by JJTree for the addition program. Every non terminal in the grammar becomes a node in the tree; the leaves carry the actual tokens (identifiers and literals).

```
Program
  VarBlock
    VarDecl (int abc = 3)
    VarDecl (int xyz = 4)
    VarDecl (int result = 0)
  CodeBlock
    AssignStmt (result)
      Expr
        Term -> Factor (abc)
        Term -> Factor (xyz)
    OutStmt
      Expr -> Term -> Factor (result)
```

Figure 3.1: AST of the simple addition CL program (text form printed by `node.dump()`).

Chapter 4

Symbol Table and Semantic Analysis

4.1 Symbol Table

The symbol table is the data structure the semantic analyser uses to keep track of every identifier the program declares. Without it the compiler cannot tell whether a variable mentioned in the code block was actually declared, or whether the type of the right hand side of an assignment matches the type of the left hand side.

We implement the symbol table in a separate Java class, `SymbolTable.java`. Internally it is a `LinkedHashMap` keyed on the identifier lexeme. Each entry stores the lexeme, the declared type, the current value (initially the literal from the `variables` block) and the source line number where the identifier was declared. A linked map is used instead of a plain hash map only so that the table prints in declaration order, which makes the console output easier to read. Since CL has a single global scope a flat map is enough; there is no need for a stack of scopes. Table 4.1 shows the symbol table contents after parsing the addition program.

Lexeme	Type	Value	Line
abc	int	3	5
xyz	int	4	6
result	int	0	7

Table 4.1: Symbol table after parsing the addition program.

4.2 Semantic Checks

The semantic analyser lives in its own class `SemanticAnalyzer.java` and walks the AST after parsing. It performs the following checks.

- **Declaration / value type agreement.** The initial value in `VarDecl` must agree with the declared type; otherwise the declaration is rejected.

-
- **Duplicate declaration.** If a variable is declared more than once in the variables block, every declaration after the first is reported as an error.
 - **Undeclared variable.** Every identifier used in the code block must be present in the symbol table.
 - **Type mismatch in assignment.** The right hand side type must be exactly equal to the left hand side type.
 - **Type mismatch in expression.** The two operands of a binary operator must have the same type. This is exactly the rule given in the assignment.
 - **Operator not allowed on a string or char.** The arithmetic operators do not apply to text values, so any use of `+`, `-`, `*` or `/` on a string or char operand is rejected.
 - **Type mismatch in condition.** The two operands of `CondExpr` must be the same numeric type.
 - **Type mismatch in switchFor.** Every case literal must have the same type as the switch variable.

All errors are accumulated in a list and only printed once at the end. If even one semantic error is present the intermediate code generation phase is skipped so we do not try to emit code for a program that does not type check.

4.3 Error Examples

The console output for two short invalid programs is shown below.

Example 1 (undeclared variable):

```
startProgram
  variables:
    int x = 1;
  code:
    y = x + 1;
endProgram
```

Output:

```
Semantic Error at line 6: identifier 'y' is not declared
```

Example 2 (type mismatch in expression, the case from the spec):

```
startProgram
```

```
variables:
  string b = "hi";
  int    c = 5;
  string a = "";
code:
  a = b + c;
endProgram
```

Output:

```
Semantic Error at line 8: type mismatch in expression:
      '+' between string and int
```

Chapter 5

Intermediate Code Generation

5.1 Overview

Intermediate code sits between the high level source and the final target code. The most common form is Three Address Code (TAC), where every instruction has at most three operands and uses temporary variables `t1`, `t2`, `t3` to break complex expressions into simple steps. Using an intermediate representation makes optimisation and target code generation much easier later on, because each TAC instruction maps to only a small handful of machine instructions and most optimisations can be done on the intermediate form without ever touching machine details.

In our project the intermediate code generator is a separate Java class, `IntermediateCodeGenerator`. It walks the same AST that the semantic analyser walks, but only after the semantic phase has reported zero errors. Each TAC instruction is stored as one `Quad` object (`op`, `arg1`, `arg2`, `result`) and the whole sequence is kept in an `ArrayList`, which is the Java backed array required by the assignment. The contents of the array are then printed in two styles: the quadruple table that matches the format in the assignment PDF, and a friendlier statement form.

5.2 Input Expression

To stay close to the example in the assignment PDF, we use the assignment from the addition program:

```
result = abc + xyz;
```

5.3 Generated Three Address Code

Walking the AST in post order, the generator emits the following TAC:

```
t1 = abc + xyz
result = t1
```

A slightly more interesting expression with mixed precedence, $r = a + b * c - d / e;$, comes out as:

```
t1 = b * c
t2 = a + t1
t3 = d / e
t4 = t2 - t3
r = t4
```

Notice how $*$ and $/$ are evaluated first because of how **Expr** and **Term** are structured in the grammar.

5.4 Quadruple Representation

The same TAC can also be written as a table of quadruples (**op**, **arg1**, **arg2**, **result**), which is the form used for later optimisation passes. Table 5.1 shows the quadruple form of the mixed precedence expression.

No	Op	Arg1	Arg2	Result
0	*	b	c	t1
1	+	a	t1	t2
2	/	d	e	t3
3	-	t2	t3	t4
4	=	t4	—	r

Table 5.1: Quadruple representation of $r = a + b * c - d / e;$.

5.5 Code for Control Flow

For **loopif** the generator emits a label, the condition, a conditional jump to the end label, the body, an unconditional jump back to the start label and finally the end label. For **switchFor** the generator first emits a dispatch table that compares the switch variable against each case literal and jumps to that case's body if the comparison is true. After the dispatch table comes an unconditional jump to the **other** label. Then for every case the generator emits a label, the case body and a jump to the common end label. Finally the **other** label and the end label themselves are emitted. Table 5.2 shows the quadruples for the factorial program.

No	Op	Arg1	Arg2	Result
0	=	5	–	n
1	=	1	–	fact
2	label	L1	–	–
3	>	n	0	t1
4	ifFalse	t1	–	L2
5	*	fact	n	t2
6	=	t2	–	fact
7	-	n	1	t3
8	=	t3	–	n
9	goto	–	–	L1
10	label	L2	–	–
11	out	fact	–	–

Table 5.2: Quadruples generated for the factorial program.

Chapter 6

Testing and Results

6.1 Testing Strategy

The compiler is interactive: when launched it shows a short banner and then waits on standard input for the user to paste or type a CL program. The program is closed off by a line that contains just `endProgram` and at that point the compiler runs every phase (scanner, parser, semantic analyser and intermediate code generator) and prints the result. For testing we wrote two groups of small CL programs and pasted each one into the compiler in turn: programs that are well formed and should be accepted, and programs that contain one specific kind of error and should be rejected with a clear, line numbered message. For the valid programs we also inspected the printed symbol table and the printed three address code to confirm they were correct.

6.2 Valid Test Cases

ID	Input	Result
V1	Addition of two integers and printing the result.	Pass
V2	Factorial of 5 computed using <code>loopif</code> .	Pass
V3	<code>switchFor</code> on an integer with three cases plus <code>other</code> .	Pass
V4	Mixed precedence expression <code>a + b * c - d / e</code> .	Pass
V5	A program that declares one variable of every type (<code>int</code> , <code>float</code> , <code>string</code> , <code>char</code>) and prints each one.	Pass

Table 6.1: Valid test cases.

6.3 Invalid Test Cases

ID	Error Type	Result
I1	Use of an undeclared identifier (y).	Semantic error at line 6
I2	Type mismatch in <code>a = b + c</code> where <code>b</code> is string and <code>c</code> is int.	Semantic error at line 8
I3	The same variable declared twice.	Semantic error at line 5
I4	Missing semicolon at the end of an assignment.	Syntax error at line 7
I5	A <code>loopif</code> nested inside another <code>loopif</code> .	Syntax error at line 9

Table 6.2: Invalid test cases.

6.4 Screenshots

The compiler is invoked from the command line with `run.bat`; it then waits for a CL program on standard input and, once the input is closed, prints (in order) the AST, the symbol table, any semantic errors and the generated three address code. Screenshots of these outputs for both the valid and invalid programs above are included in Appendix C.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

The project has produced a working compiler front end for CL together with a three address code generator. The lexical analyser and the parser are generated by JavaCC from a single `CL.jjt` file. The symbol table and the semantic analyser are separate hand written Java classes, as the project specification required. The intermediate code generator emits quadruples into a Java array and prints them in both the table form and a readable statement form. The compiler accepts every program in our valid test set and reports a clear, line numbered error for every program in our invalid set.

7.2 Learning Outcomes

Writing the grammar in a form JavaCC was willing to accept forced us to think carefully about left recursion, lookahead and operator precedence – topics that look harmless on paper but become real engineering problems the moment a real parser is involved. Implementing the symbol table made the difference between syntactic and semantic correctness completely concrete: a program can parse perfectly and still be nonsense. Generating three address code by walking the AST showed us why an intermediate representation is so useful, even for a language as small as CL.

7.3 Future Work

- A simple optimisation pass over the quadruples (constant folding, dead code elimination, copy propagation).
- A code generator that turns the quadruples into x86 64 assembly or into bytecode for a small virtual machine.

-
- A small GUI that shows the source, the tokens, the AST, the symbol table and the TAC side by side.
 - Extending the language with arrays, user defined functions and proper lexical scoping.

References

1. Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. *Compilers: Principles, Techniques, and Tools (Dragon Book)*, 2nd ed., Pearson, 2007.
2. JavaCC Project Documentation. <https://javacc.github.io/javacc/>
3. JJTree Reference Documentation. <https://javacc.github.io/javacc/tutorials/jjtree.htm>
4. Course slides and lecture notes, CSC323 Compiler Construction, Spring 2026.
5. Appel, A. W. *Modern Compiler Implementation in Java*, Cambridge University Press.

Chapter A

Source Code

Selected portions of the .jjt grammar file and the symbol table are listed below. The full code is included in the submitted zip file.

```
// CL.jjt (excerpt)
options { MULTI = true; NODE_DEFAULT_VOID = true; STATIC = false; }
```

```
PARSER_BEGIN(CLParser)
package cl;
public class CLParser { /* main goes here */ }
PARSER_END(CLParser)
```

```
SKIP : { " " | "\t" | "\n" | "\r" }
SKIP : { <"//" (~["\n","\r"])*> }
```

```
TOKEN : {
    < T_START      : "startProgram" >
  | < T_END        : "endProgram"   >
  | < T_VARS       : "variables"    >
  | < T_CODE       : "code"         >
  | < T_INT        : "int"          >
  | < T_FLOAT_T    : "float"        >
  | < T_STR_T      : "string"       >
  | < T_CHAR_T     : "char"         >
  | < T_LOOPIF     : "loopif"       >
  | < T_HOLDS      : "holds"        >
  | < T_ENDLOOP    : "endloop"      >
  | < T_OUTSTRING  : "outString"    >
  | < IDENT        : ["a"-"z","A"-"Z","_"]
```

```

        (["a"-"z","A"-"Z","0"-"9","_"])* >
| < INTEGER      : (["0"-"9"])+ >
| < FLOATING     : (["0"-"9"])+ "." (["0"-"9"])+ >
| < STRLIT       : "\" ( ~["\""] ) * "\" >
| < CHARLIT      : "'" ( ~["'"] ) "'" >
}

// SymbolTable.java (excerpt)
public class SymbolTable {

    public static class Entry {
        public final String lexeme;
        public final String type;
        public String      value;
        public final int   line;

        public Entry(String lexeme, String type, String value, int line) {
            this.lexeme = lexeme;
            this.type    = type;
            this.value   = value;
            this.line    = line;
        }
    }

    private final Map<String, Entry> table = new LinkedHashMap<>();

    public boolean declare(String lexeme, String type,
                           String value, int line) {
        if (table.containsKey(lexeme)) return false;    // duplicate
        table.put(lexeme, new Entry(lexeme, type, value, line));
        return true;
    }

    public Entry lookup(String lexeme) { return table.get(lexeme); }
}

```

Chapter B

Sample Input Programs

Three of the programs we used during testing are listed below. Each one was pasted directly into the compiler at runtime, terminated by the `endProgram` line that ends every CL program.

```
// P1 : simple arithmetic
startProgram
  variables:
    int a = 10; int b = 20; int c = 0;
  code:
    c = a + b;
    outString(c);
endProgram
```

```
// P2 : factorial
startProgram
  variables:
    int n = 5; int f = 1;
  code:
    loopif n > 0 holds
      f = f * n;
      n = n - 1;
    endloop
    outString(f);
endProgram
```

```
// P3 : switchFor
startProgram
  variables:
    int g = 2; int r = 0;
```

```
code:
  switchFor (g)
    case 1 : r = 100;
    case 2 : r = 200;
    other  : r = 0;
  endswitchFor
  outString(r);
endProgram
```

Chapter C

Extra Screenshots

Additional console outputs and AST screenshots are placed here in the final submission.

Mandatory Rules

- LaTeX PDF only
- Original screenshots required
- No plagiarism
- Use of AI is strictly prohibited
- Viva based on report